

Banco de dados do Lume

Lume para Windows · SQLite embutido no Java

Esquema, dicionário de dados, ciclo de vida, busca em texto completo com FTS5, integridade, consultas, backup e evolução.

Copyright (©) 2026 Erik Pimenta e Clara Portes

Versão 2.0.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral **3**

Por que SQLite 3

Como o banco é aberto 3

Arquivos na pasta de dados 3

Modelo de dados **5**

Dicionário de dados **7**

documents 7

document_content 8

tags e document_tags 8

collections e collection_items 8

reading_states 9

annotations 9

jobs 9

settings 10

Ciclo de vida de um documento **11**

Busca em texto completo **12**

A tabela 12

Da caixa de busca para o MATCH 12

Ranking e trecho 13

Integridade e transações **14**

Transações 14

Consultas úteis **15**

Backup, restauração e manutenção **16**

Backup 16

Restauração e mudança de computador 16

Manutenção 16

Evolução do esquema e versão web **17**

Criar uma migração 17

Comparação com a versão web 17

CAPÍTULO 1

Visão geral

O Lume para Windows guarda tudo num único arquivo SQLite, o `lume.db`, dentro da pasta de dados. Este manual descreve o esquema, as regras que o banco garante sozinho, a busca em texto completo e como fazer backup, consultar e evoluir o banco.

Por que SQLite

- **Zero instalação:** o SQLite vem dentro do próprio Lume, no driver JDBC `sqlite-jdbc` 3.53 (Xerial), que já traz o motor do SQLite compilado com FTS5 e JSON. Não há servidor, porta nem senha.
- **Um usuário, um computador:** o Lume desktop não tem contas nem compartilhamento, então a concorrência é baixa e o SQLite sobra.
- **Portátil:** copiar a pasta de dados leva a biblioteca inteira para outro computador ou para um pendrive.
- **Busca embutida:** o FTS5 faz a busca em texto completo sem acentos, com ranking BM25 e trechos destacados, no lugar do tsvector e dos trigramas do PostgreSQL da versão web.
- **Compatível com a versão anterior:** o esquema é o mesmo da versão em Electron (esquema 2). Uma biblioteca criada por ela abre direto na versão em Java, sem conversão.

Como o banco é aberto

A classe `Database` (pacote `io.github.ravelroot.lume.db`) abre uma conexão JDBC só, `jdbc:sqlite:<pasta>/lume.db`, e liga o modo de trabalho:

```
PRAGMA journal_mode = WAL;    -- leituras não esperam escritas; grava lume.db-wal ao lado
PRAGMA synchronous = NORMAL;  -- seguro com WAL e bem mais rápido que FULL
PRAGMA foreign_keys = ON;     -- o SQLite só aplica chaves estrangeiras com isto ligado
PRAGMA busy_timeout = 5000;   -- espera até 5 s se o arquivo estiver ocupado
```

Em seguida `migrate()` compara `PRAGMA user_version` com a lista `Migrations.ALL` e aplica as que faltam, cada uma numa transação. Se o banco tiver uma versão maior que a do aplicativo (foi aberto por um Lume mais novo), a abertura é recusada com uma mensagem clara, para não estragar dados. Ao fechar (quando o Spring encerra o bean), o Lume faz `PRAGMA wal_checkpoint(TRUNCATE)`, devolvendo tudo para o `lume.db`.

As consultas passam pelo `JdbcTemplate` do Spring, sobre um `SingleConnectionDataSource`. Uma trava (`ReentrantLock`) garante que a API, o pipeline e os eventos usem a conexão um de cada vez.

Arquivos na pasta de dados

Caminho	Conteúdo
`lume.db`	O banco (fonte da verdade de tudo que não é arquivo)
`lume.db-wal` · `lume.db-shm`	Diário do modo WAL; somem ou ficam vazios quando o Lume fecha
`documentos/<id>/original.pdf|docx`	Cópia do arquivo importado
`documentos/<id>/capa.jpg`	Capa (primeira página ou capa tipográfica)

Caminho	Conteúdo
`documentos/<id>/leitura.html` · `media/`	HTML de leitura higienizado e imagens dos DOCX
`tmp/`	Cópias em andamento durante a importação
`lume.lock` · `instancia.json`	Trava de instância única e porta/chave do Lume aberto
`lume.log`	Registro do aplicativo (útil para suporte)

BANCO E ARQUIVOS ANDAM JUNTOS

O `id` do documento é o nome da pasta em `documentos/`. Ao excluir um documento, o Lume apaga a linha (e, em cascata, tudo que depende dela) e depois a pasta. Nunca apague pastas à mão com o Lume aberto.

CAPÍTULO 2

Modelo de dados

Onze tabelas giram em torno de `documents`. Todas as chaves estrangeiras usam `ON DELETE CASCADE`: apagar um documento apaga o conteúdo, a leitura, as anotações, as etiquetas ligadas, os itens de coleção e o histórico de etapas.

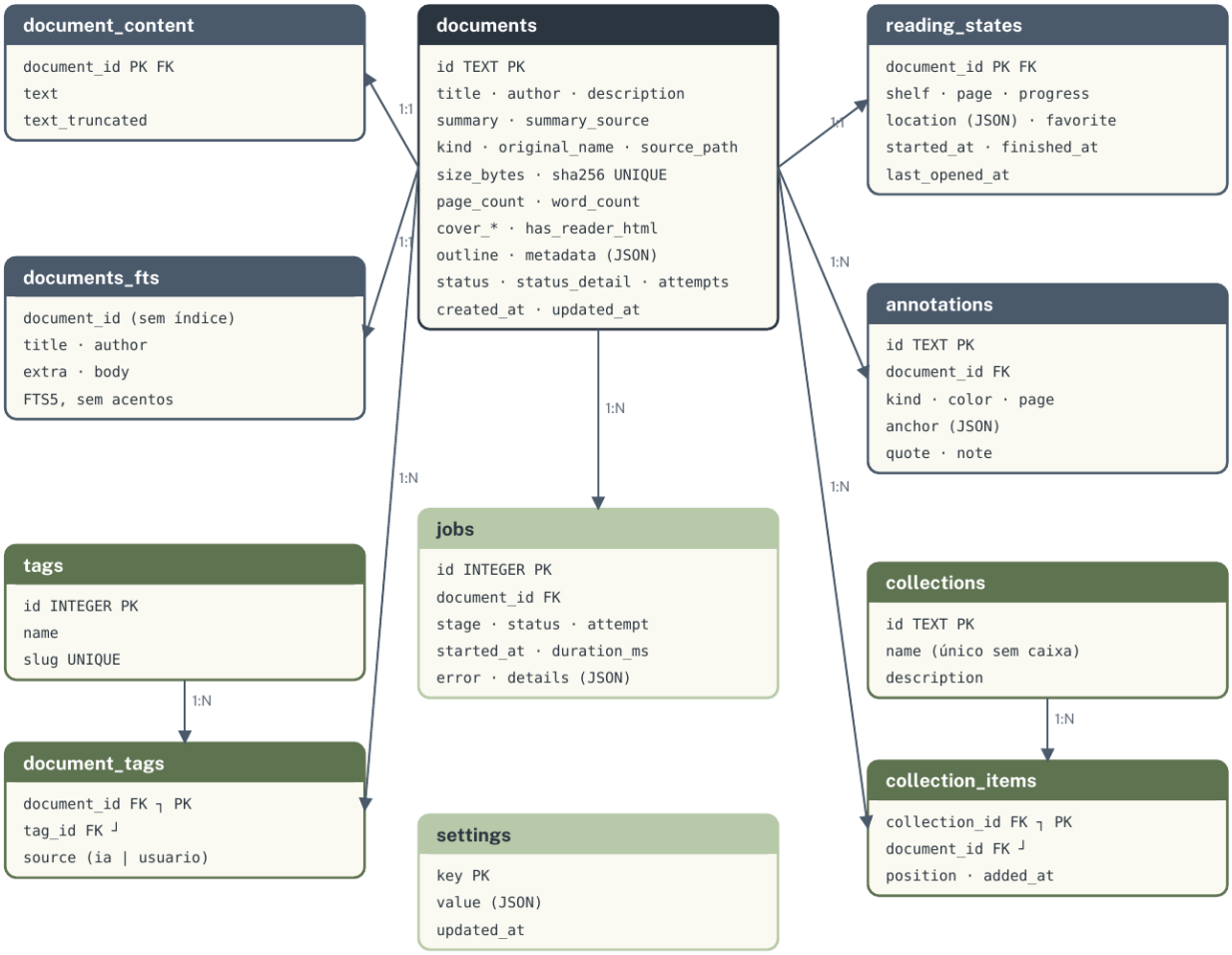


Tabela	Uma linha é	Chave
`documents`	um arquivo importado	`id` (UUID em texto)
`document_content`	o texto extraído de um documento (separado para as listagens ficarem leves)	`document_id`
`documents_fts`	a entrada do documento no índice de busca (tabela virtual FTS5)	`rowid` interno
`tags` · `document_tags`	uma etiqueta · a ligação etiqueta–documento, com a origem	`id` · (documento, etiqueta)
`collections` · `collection_items`	uma coleção · um documento numa coleção, com a posição	`id` · (coleção, documento)

Tabela	Uma linha é	Chave
`reading_states`	o estado de leitura de um documento	`document_id`
`annotations`	um destaque, nota ou marcador	`id` (UUID)
`jobs`	uma etapa do pipeline (leitura ou descrição) de um documento	`id` inteiro
`settings`	uma preferência (perfil, IA, janela)	`key`

Datas são texto ISO 8601 em UTC (`2026-10-20T21:44:50.315Z`, gerado por `Times.now()`), o que ordena corretamente como texto. Booleanos são `INTEGER` 0 ou 1 com `CHECK`. Campos estruturados (sumário, metadados, posição de leitura, âncora da anotação, detalhes da etapa, preferências) são JSON em texto, validados com `CHECK (json_valid(...))` e lidos com o Jackson.

CAPÍTULO 3

Dicionário de dados

Coluna por coluna, com as regras que o banco aplica e as que a aplicação garante.

documents

Coluna	Tipo	Regra
`id`	TEXT PK	UUID v4 gerado na importação; nome da pasta do documento
`title` · `title_sort`	TEXT	Título (até 300) e a versão minúscula sem acentos, usada para ordenar e na busca por trecho do título
`author` · `author_sort`	TEXT	Autor (até 200) e versão para ordenar
`description`	TEXT	Descrição escrita pela pessoa (até 5000)
`metadata_locked`	INTEGER 0/1	1 depois que a pessoa edita título ou autor: os metadados do arquivo não sobrescrevem mais
`summary` · `summary_source` · `summary_model` · `summary_at`	TEXT	Descrição por IA, quem escreveu (`local`, `anthropic`, `openai`), modelo e data
`kind`	TEXT	`CHECK IN ('pdf', 'docx')`; decidido pelo conteúdo, não pela extensão
`original_name` · `source_path`	TEXT	Nome e caminho de onde o arquivo foi importado (só informativo)
`size_bytes`	INTEGER	`CHECK (size_bytes > 0)`; até 500 MB por regra da aplicação
`sha256`	TEXT UNIQUE	Impressão digital do conteúdo: impede o mesmo arquivo duas vezes
`page_count` · `word_count` · `language`	INTEGER · TEXT	Preenchidos na leitura (no DOCX, páginas do app.xml ou estimativa de 350 palavras por página)
`cover_width` · `cover_height` · `cover_color`	INTEGER · TEXT	Tamanho da capa e cor média (ou o tom da paleta, nas capas geradas)
`cover_generated` · `cover_version`	INTEGER	Capa tipográfica? Versão incrementada a cada capa nova (vai na URL para furar o cache)
`has_reader_html`	INTEGER 0/1	Existe `leitura.html` (DOCX)
`outline`	TEXT JSON	Sumário: `[{title, page, level, anchor}]`, até 500 itens
`metadata`	TEXT JSON	Assunto, palavras-chave, datas, programa de origem, tamanho da página, avisos da leitura
`status` · `status_detail` · `attempts`	TEXT · INTEGER	`CHECK IN ('queued','extracting','describing','ready','failed')`; mensagem para a pessoa; tentativas da etapa atual

Coluna	Tipo	Regra
`created_at` · `updated_at` · `processed_at`	TEXT	Importação, última mudança e primeira vez que ficou pronto

Índices: `created_at DESC` (ordem padrão), `status` (filtros e recuperação) e `title_sort` (ordem por título).

document_content

Coluna	Tipo	Regra
`document_id`	TEXT PK FK	Um texto por documento
`text`	TEXT	Texto limpo, até 2.000.000 de caracteres; páginas do PDF separadas por `\f`
`text_truncated`	INTEGER 0/1	O texto passou do limite e foi cortado
`updated_at`	TEXT	Última leitura

tags e document_tags

Coluna	Tipo	Regra
`tags.id`	INTEGER PK	Identificador interno
`tags.name` · `tags.slug`	TEXT · TEXT UNIQUE	Nome em minúsculas (até 40) e a forma sem acentos (`programacao`), que garante uma etiqueta por ideia
`document_tags.document_id` · `tag_id`	FK · FK	Chave primária composta: a mesma etiqueta uma vez por documento
`document_tags.source`	TEXT	`CHECK IN ('ia', 'usuario')`: a IA só troca as dela; a pessoa substitui todas

Etiquetas sem nenhum documento são apagadas na mesma transação que as deixou órfãs.

collections e collection_items

Coluna	Tipo	Regra
`collections.id`	TEXT PK	UUID
`collections.name`	TEXT	`CHECK (length(trim(name)) > 0)`; índice único em `lower(name)`
`collections.description`	TEXT	Até 300 caracteres
`collection_items.position`	INTEGER	Ordem na coleção (1, 2, 3…); novos entram no fim
`collection_items.added_at`	TEXT	Desempate da ordem

reading_states

Coluna	Tipo	Regra
`document_id`	TEXT PK FK	No máximo um estado por documento; criado na primeira interação
`shelf`	TEXT	`CHECK IN ('', 'para_ler', 'lendo', 'lido')`
`page`	INTEGER	`CHECK (page >= 1)`; limitada ao total de páginas pela aplicação
`progress`	REAL	`CHECK (progress BETWEEN 0 AND 1)`
`location`	TEXT JSON	Posição no DOCX: `{"block": 42}` (índice do bloco `data-b`)
`favorite`	INTEGER 0/1	Favorito
`started_at` · `finished_at` · `last_opened_at`	TEXT	Primeira vez em “Lendo”, quando foi para “Lido” (zera ao sair) e última abertura no leitor

annotations

Coluna	Tipo	Regra
`kind`	TEXT	`CHECK IN ('note', 'highlight', 'bookmark')`; nota exige texto
`page`	INTEGER	Página (PDF) ou nulo (DOCX)
`anchor`	TEXT JSON	PDF: `{"rects": [[x, y, w, h], ...]}` em frações da página; DOCX: `{"block": n}`
`quote` · `note`	TEXT	Trecho destacado (até 4000) e nota (até 8000)
`color`	TEXT	`CHECK IN ('salvia', 'rose', 'ardosia', 'camurca')`, as cores de destaque da paleta (a migração 2 converte as cores antigas)

Índice `(document_id, page)`. A aplicação limita a 2000 anotações por documento.

jobs

Coluna	Tipo	Regra
`stage`	TEXT	`CHECK IN ('extract', 'describe')`
`status`	TEXT	`CHECK IN ('running', 'succeeded', 'retrying', 'failed')`
`attempt`	INTEGER	Número da tentativa

Coluna	Tipo	Regra
`started_at` · `finished_at` · `duration_ms`	TEXT · INTEGER	Tempo de cada etapa (médias da tela Sistema)
`error` · `details`	TEXT · JSON	Motivo da falha ou aviso; páginas, palavras, provedor, modelo, se caiu para o resumo local

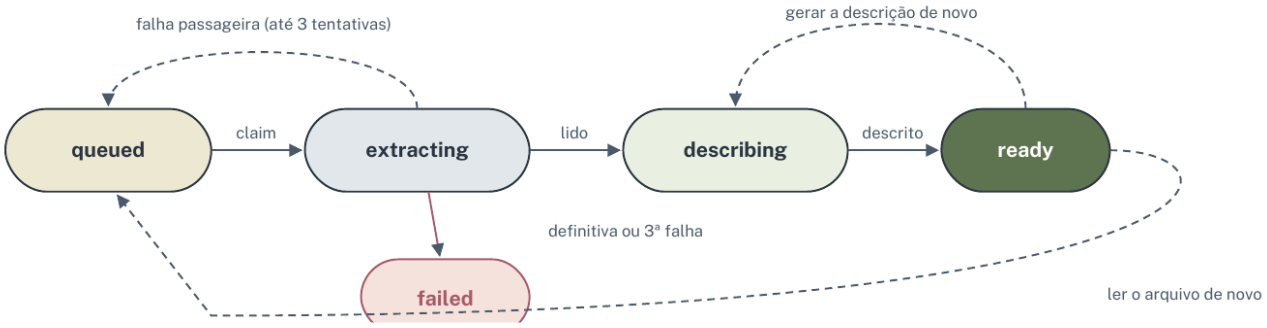
settings

Chave	Valor (JSON)
`profile`	`{name, theme, reader_prefs: {theme, fontSize}}`
`ai`	`{provider, model, base_url, api_key, timeout_s, max_input_chars}` ; `api_key` é `dpapi:` + Base64 do texto cifrado pelo cofre do Windows (DPAPI, via JNA). Chaves `enc:` da versão em Electron não abrem no Java: informe a chave de novo em Preferências
`window`	`{x, y, width, height, maximized}` da janela, usados ao abrir o Edge em modo aplicativo

CAPÍTULO 4

Ciclo de vida de um documento

A coluna `status` é uma pequena máquina de estados. Quem muda o estado é sempre o `PipelineService`, numa transação, e cada mudança vira um evento (SSE) para a tela.



Transição	Onde	O que é gravado
importação → `queued`	`LibraryService.importFile`	Linha nova, arquivo em `documentos/<id>`, entrada no índice de busca
`queued` → `extracting`	`PipelineService.claim`	`attempts + 1`; só pega quem está em estado esperado (pedido repetido não faz nada)
`extracting` → `describing`	`applyExtraction`	Páginas, palavras, sumário, metadados, capa (`cover_version + 1`), texto em `document_content`, índice de busca; `attempts = 0`
`describing` → `ready`	`runDescribe`	Descrição, origem, modelo, idioma, etiquetas da IA, `processed_at` na primeira vez
falha passageira → `queued`	`runExtract`	`status_detail` “Nova tentativa em 5 s”; etapa `retrying`
falha definitiva → `failed`	`fail`	`status_detail` com a mensagem; etapa `failed`

RECUPERAÇÃO AO ABRIR

Quando o Spring termina de subir (`ApplicationReadyEvent`), `recover()` marca como `failed` as etapas que ficaram `running`, devolve para a fila de leitura os documentos em `queued` ou `extracting` e para a fila de descrição os que estavam em `describing`.

CAPÍTULO 5

Busca em texto completo

A busca usa uma tabela virtual FTS5 com uma linha por documento, reconstruída sempre que algo pesquisável muda.

A tabela

```
CREATE VIRTUAL TABLE documents_fts USING fts5 (
  document_id UNINDEXED,   -- liga a linha ao documento (não é pesquisado)
  title,                   -- peso 10
  author,                  -- peso 5
  extra,                   -- descrição por IA + sua descrição + etiquetas, peso 3
  body,                    -- os primeiros 300.000 caracteres do texto, peso 1
  tokenize = 'unicode61 remove_diacritics 2'
);
```

O tokenizador `unicode61` com `remove_diacritics 2` separa palavras por Unicode e tira os acentos tanto do texto quanto da consulta: “reunião”, “reuniao” e “REUNIÃO” são a mesma coisa.
`SearchService.refreshSearch(id)` apaga e reinsere a linha do documento; ele roda na importação, depois da leitura, depois da descrição, ao editar título, autor ou descrição e ao trocar etiquetas, sempre dentro da mesma transação da mudança.

Da caixa de busca para o MATCH

Você digita	Vira	Por quê
`contrato aluguel`	`"contrato" AND "aluguel"`	todas as palavras, aceitando o começo
`"reunião de planejamento"`	`"reunião de planejamento"`	frase exata
`python -flask`	`"python" NOT "flask"`	exclusão
`a"b OR`	`"a" AND "b" AND "or"*`	aspas e operadores digitados viram texto: a consulta nunca quebra

Além do `MATCH`, a listagem aceita documentos cujo `title_sort` contenha o texto digitado (`LIKE`), para achar pedaços no meio de palavras do título.

Ranking e trecho

```
SELECT d.*
FROM documents d
LEFT JOIN (SELECT document_id, bm25(documents_fts, 0, 10.0, 5.0, 3.0, 1.0) AS rank
           FROM documents_fts WHERE documents_fts MATCH :consulta) f ON f.document_id = d.id
WHERE (f.document_id IS NOT NULL OR d.title_sort LIKE :trecho ESCAPE '\')
ORDER BY (f.rank IS NULL), f.rank, d.created_at DESC
LIMIT :tamanho OFFSET :inicio;

-- trecho com a palavra destacada, só para os documentos da página
SELECT document_id, snippet(documents_fts, 4, char(2), char(3), ' … ', 26)
FROM documents_fts WHERE documents_fts MATCH :consulta AND document_id IN (...);
```

O BM25 do FTS5 devolve números menores para os melhores resultados; os pesos fazem o título valer mais que o texto. O trecho usa os caracteres de controle 2 e 3 como marcadores: o texto é escapado para HTML e só então eles viram `<mark>`, então nada do documento vira HTML na tela.

CAPÍTULO 6

Integridade e transações

As regras importantes ficam no próprio banco, não só no código: mesmo um script que escreva direto no `lume.db` não consegue gravar um estado inválido.

Regra	Como o banco garante
Um arquivo, um documento	`sha256 UNIQUE` em `documents`; importações simultâneas do mesmo arquivo: a segunda vira "duplicado"
Valores dentro do domínio	`CHECK` em `kind`, `status`, `shelf`, `color`, `stage`, booleanos, `progress`, `page`, `size_bytes`
JSON válido	`CHECK (json_valid(...))` em `outline`, `metadata`, `location`, `anchor`, `details`, `value`
Sem órfãos	`FOREIGN KEY ... ON DELETE CASCADE` em todas as tabelas filhas, com `PRAGMA foreign_keys = ON`
Nomes de coleção únicos	`UNIQUE INDEX ON collections (lower(name))`
Uma ligação por par	Chaves primárias compostas em `document_tags` e `collection_items`

Transações

`Database.transaction(...)` recebe um `Runnable` ou `Supplier`: a primeira chamada abre `BEGIN IMMEDIATE` (reserva a escrita na hora, evitando conflito no meio) e chamadas aninhadas viram `SAVEPOINT`. Se qualquer coisa lançar exceção, tudo é desfeito (`ROLLBACK` ou `ROLLBACK TO`). Exemplos: criar o documento e mover o arquivo para a pasta dele; gravar o resultado da leitura, o texto e o índice de busca; trocar as etiquetas e apagar as órfãs.

```
db.transaction(() -> {
    db.update("INSERT INTO documents (...) VALUES (...)", ...);
    Files.move(tmp, pasta.resolve("original." + kind));   // falhou? a linha também volta
    search.refreshSearch(id);                             // SAVEPOINT aninhado
});
```

> CONCORRÊNCIA
>
> Só o processo do Lume escreve no banco, por uma conexão só, protegida por uma trava (`ReentrantLock`). Os fios da API (Tomcat), do pipeline (`ExecutorService`) e dos eventos esperam a vez, então não há disputa, e o modo WAL deixa leituras externas correrem enquanto o pipeline grava.

CAPÍTULO 7

Consultas úteis

Para abrir o banco fora do Lume, feche o aplicativo e use o `sqlite3` ou o DB Browser for SQLite. Faça uma cópia antes de alterar algo.

```
-- Quantos documentos em cada estado
SELECT status, count(*) FROM documents GROUP BY status;

-- Espaço ocupado pelos originais, por tipo
SELECT kind, count(*) AS docs, round(sum(size_bytes) / 1048576.0, 1) AS mb FROM documents GROUP BY
kind;

-- O que estou lendo, do mais recente para o mais antigo
SELECT d.title, r.page, d.page_count, round(r.progress * 100) AS pct, r.last_opened_at
FROM reading_states r JOIN documents d ON d.id = r.document_id
WHERE r.shelf = 'lendo' ORDER BY r.last_opened_at DESC;

-- Etiquetas mais usadas
SELECT t.name, count(*) AS n FROM document_tags dt JOIN tags t ON t.id = dt.tag_id
GROUP BY t.id ORDER BY n DESC LIMIT 20;

-- Todas as anotações de um documento, em ordem de página
SELECT page, kind, color, quote, note FROM annotations
WHERE document_id = (SELECT id FROM documents WHERE title LIKE 'Guia prático%')
ORDER BY page, created_at;

-- Tempo médio de leitura e descrição nos últimos 7 dias
SELECT stage, count(*), round(avg(duration_ms)) AS ms FROM jobs
WHERE status = 'succeeded' AND started_at >= strftime('%Y-%m-%dT%H:%M:%fZ', 'now', '-7 days')
GROUP BY stage;

-- Documentos que caíram para o resumo local
SELECT d.title, j.error FROM jobs j JOIN documents d ON d.id = j.document_id
WHERE j.stage = 'describe' AND json_extract(j.details, '$.fallback') = 1;

-- Busca direto no índice
SELECT d.title FROM documents_fts f JOIN documents d ON d.id = f.document_id
WHERE documents_fts MATCH '"cooperativa"*' ORDER BY bm25(documents_fts, 0, 10, 5, 3, 1);
```

CAPÍTULO 8

Backup, restauração e manutenção

A biblioteca é a pasta de dados inteira: o banco e a pasta `documentos/`.

Backup

1. Feche o Lume (a janela fecha e o processo encerra sozinho; o diário WAL é incorporado ao `lume.db`).
2. Copie a pasta de dados inteira: o banco sozinho não basta, os arquivos e capas estão em `documentos/`.
3. Para um backup com o Lume aberto, use o comando `.backup` do `sqlite3` (`sqlite3 lume.db ".backup copia.db"`), que faz uma cópia consistente, e copie `documentos/` depois.

Restauração e mudança de computador

Ponha a pasta de volta em `%APPDATA%\Lume\Biblioteca` (ou como `Lume-dados\Biblioteca` ao lado do executável, ou no caminho de `LUME_DATA_DIR`). A chave de API cifrada só abre no mesmo usuário do Windows: em outro computador, cadastre de novo em Preferências. Bibliotecas da versão 1.x (Electron) abrem direto, porque a pasta e o esquema são os mesmos.

Manutenção

Comando	Quando usar
`PRAGMA integrity_check;`	Depois de uma queda de energia ou se algo parecer estranho: deve responder `ok`
`PRAGMA foreign_key_check;`	Conferir se ninguém deixou órfãos editando o banco à mão
`INSERT INTO documents_fts(documents_fts) VALUES ('optimize');`	Compactar o índice de busca depois de muitas importações
`VACUUM;`	Devolver ao disco o espaço de muitos documentos apagados (com o Lume fechado)
`DELETE FROM jobs WHERE started_at < date('now', '-180 days');`	Limpar o histórico antigo de etapas

CAPÍTULO 9

Evolução do esquema e versão web

Como mudar o banco sem perder dados e como ele se compara ao PostgreSQL da versão web.

Criar uma migração

1. Acrescente um texto SQL no fim da lista `ALL` em `src/main/java/io/github/ravelroot/lume/db/Migrations.java`. Nunca edite migrações já publicadas.
2. Prefira mudanças aditivas: `ALTER TABLE ... ADD COLUMN` com `DEFAULT`, tabelas e índices novos.
3. Para mudar o tipo ou as restrições de uma coluna, siga o roteiro do SQLite: criar a tabela nova, copiar com `INSERT ... SELECT`, apagar a antiga e renomear, tudo na mesma migração (a migração 2 faz exatamente isso com as cores das anotações).
4. Rode `mvn test`: o `DatabaseTest` abre um banco vazio, confere as tabelas, abre de novo para garantir que nada é reaplicado e testa a migração de um banco no esquema 1.

```
// src/main/java/io/github/ravelroot/lume/db/Migrations.java
static final String V3 = """
        ALTER TABLE reading_states ADD COLUMN seconds_read INTEGER NOT NULL DEFAULT 0;
        """;   // 3: exemplo — tempo total de leitura

public static final List<String> ALL = List.of(V1, V2, V3);
```

Comparação com a versão web

Versão web (PostgreSQL)	Lume para Windows (SQLite)
`accounts_user`, `accounts_apitoken`	Não existem: um usuário só; o perfil fica em `settings`
`library_document.owner_id` e `visibility`	Não existem: tudo é de quem usa o computador
`uuid` nativo	UUID em `TEXT`
`jsonb`	`TEXT` com `CHECK (json_valid(...))` e `json_extract` nas consultas
`tsvector` com pesos A/B/C, configuração `pt_unaccent`, `ts_headline`	FTS5 com `remove_diacritics 2`, pesos no `bm25()` e `snippet()`
Trigramas (`pg_trgm`) no título	`LIKE` no `title_sort` sem acentos
Redis Streams, ZSET de atraso e pub/sub	Fila em memória com `ExecutorService` e `ScheduledExecutorService`; eventos por Server-Sent Events; histórico em `jobs`
`SELECT ... FOR UPDATE SKIP LOCKED`	`BEGIN IMMEDIATE` e uma conexão só, protegida por trava
ORM e migrações do Django	`JdbcTemplate` do Spring, lista `Migrations.ALL` e `PRAGMA user_version`